Python Fundamentals Chapter 02

for Engineering Students

Identifiers · Variables · Functions · Classes · Objects
Keywords · Built-in Names

Department of Computer Science & Engineering

Session Objectives

Understand what Identifiers are and the rules for naming them

Build Classes and understand Object-Oriented principles

Learn how Variables store and manage data in Python

Memorize Python's reserved Keywords and their purpose

Define and use Functions to write reusable logic

Explore Python's powerful Built-in names and functions

What is an Identifier?

An identifier is a name given to a program element — variable, function, class, module, or object.

Naming Rules

- Must start with a letter (a–z, A–Z) or underscore _
- Can contain letters, digits (0–9), and underscores
- Case-sensitive: myVar ≠ MyVar ≠ MYVAR
- Cannot be a Python keyword
- No spaces or special characters allowed

```
# ✅ Valid Identifiers
student_name = 'Alice'

_count = 42

TotalMarks2024 = 95
# ❌ Invalid Identifiers

2marks   # starts with digit

my-var    # hyphen not allowed

class     # reserved keyword
```

Variables in Python

A variable is a named memory location that stores a value. Python is dynamically typed — no need to declare type explicitly.

Integer	Float	String	Boolean
`age = 21`	`pi = 3.14`	`name = "Alice"`	`is_on = True`

```
# Multiple assignment & type() function

x, y, z = 10, 20, 30

a = b = c = 0

print(type(x))  # <class 'int'>
```

Key Points

- No declaration needed
- Type changes at runtime
- Everything is an object
- Use snake_case naming

Functions in Python

A function is a named block of reusable code that performs a specific task. Defined with the def keyword.

```
def calculate_area(length, width):

    """Returns area of a rectangle"""
    area = length * width

    return area



# Function Call

result = calculate_area(5, 3)

print(result)  # Output: 15
```

Parameters

Inputs to the function (length, width)

Docstring

Documentation string describing the function

return

Sends result back to caller

Call

Execute function with actual arguments

Functions promote code reuse, modularity, and readability

Classes — Blueprint for Objects

A class is a blueprint (template) for creating objects. It bundles data (attributes) and behavior (methods) together.

```
class Student:

    def __init__(self, name, roll):

        self.name = name

        self.roll = roll


    def display(self):

        print(f"{self.name} - Roll: {self.roll}")
```

`class` keyword

Declares a new class named Student

`__init__`

Constructor — runs when object is created

`self`

Reference to the current instance

Attributes

name, roll — data stored in each object

Method

display() — behavior of the class

Instance Objects

CLASS (Blueprint)

```
class Student:
  name
  roll
  display()
```

instantiate

OBJECT s1

```
name = "Alice"
roll = "101"

display()
```

OBJECT s2

```
name = "Bob"
roll = "102"

display()
```

```
# Creating instance objects
s1 = Student('Alice', 101)   s2 = Student('Bob', 102)
s1.display()   # Alice - Roll: 101     s2.display()   # Bob - Roll: 102
```

Python Keywords — Reserved Words

Python has 35 reserved keywords that cannot be used as identifiers:

`False`	`None`	`True`	`and`	`as`	`assert`	`async`
`await`	`break`	`class`	`continue`	`def`	`del`	`elif`
`else`	`except`	`finally`	`for`	`from`	`global`	`if`
`import`	`in`	`is`	`lambda`	`nonlocal`	`not`	`or`
`pass`	`raise`	`return`	`try`	`while`	`with`	`yield`

Control Flow

OOP / Logic

Module / Scope

Values / Async

Keywords — Usage Examples

if / elif / else

```
if score >= 90:
    grade = 'A'
elif score >= 75:
    grade = 'B'
else:
    grade = 'C'
```

for / while / break

```
for i in range(5):
    if i == 3: break
    print(i)   # 0 1 2
```

def / return / lambda

```
def square(x): return x**2
double = lambda n: n * 2
print(square(4))   # 16
```

try / except / finally

```
try:
    x = int(input())
except ValueError:
    print('Invalid!')
finally:
    print('Done')
```

Tip: Use import keyword to check all keywords — import keyword; print(keyword.kwlist)

Python Built-in Names

Built-in names are pre-defined and always available — no import needed. They live in the __builtins__ module.

Input / Output

```
print()   input()   open()
format()
```

Type Conversion

```
int()   float()   str()   bool()
list()   tuple()   dict()
set()
```

Inspection & Info

```
type()   len()   id()   dir()
help()   isinstance()
```

Math & Logic

```
abs()   max()   min()   sum()
round()   pow()   divmod()
```

Iteration & Sequence

```
range()   enumerate()   zip()
map()   filter()   sorted()
reversed()
```

Built-in Types

```
int   float   str   list   dict
tuple   set   bool   bytes
```

Built-in Names — Usage Examples

print() / input() / len()

```
name = input('Enter name: ')
print('Hello,', name)
marks = [85, 90, 78, 92]
print(len(marks))   # 4
```

type() / int() / str()

```
x = '42'
print(type(x))     # <class 'str'>
y = int(x)
print(type(y))     # <class 'int'>
```

range() / enumerate() / zip()

```
for i, val in enumerate(['a','b','c']):
    print(i, val)  # 0 a, 1 b, 2 c

for a, b in zip([1,2], ['x','y']):
    print(a, b)   # 1 x, 2 y
```

max() / min() / sorted()

```
nums = [3, 1, 4, 1, 5, 9, 2]
print(max(nums))    # 9
print(min(nums))    # 1
print(sorted(nums)) # [1,1,2,3,4,5,9]
```

Session Summary

Identifiers

Names for program elements — follow strict rules

Instance Objects

Concrete instances created from a class

Variables

Named storage; dynamically typed in Python

Keywords (35)

Reserved words — cannot be used as identifiers

Functions

Reusable code blocks defined with def

Built-in Names

Pre-defined functions always available — no import

Classes

Blueprints that bundle data and methods

Practice these concepts daily — Python mastery comes with consistent hands-on coding! 🚀